

Piazza Cloud Computing Coursework

Complete beginner-friendly build, test, Docker, Google Cloud VM and Kubernetes tutorial

Technology: Node.js (JavaScript), Express, MongoDB Atlas, Postman, Docker, Google Cloud Compute Engine and Google Kubernetes Engine.

This guide is written to accompany the included commented project files. Replace placeholders with your own student details, cloud project identifiers, screenshots and evidence before submission.

Important correction: The coursework does not ask for Java. It recommends Node.js, which means server-side JavaScript. The supplied tutorial and project therefore use JavaScript, Express and MongoDB.

Prepared from the seven-page Piazza coursework brief and the warestack/cc tutorial repository. Cloud console labels may change slightly over time; the underlying commands and resource types remain the same.

1. What you must submit and how the marks are earned

The brief requires a cloud Software as a Service called Piazza. It behaves like a small topic-based social network: authenticated users publish posts, browse by topic, like, dislike and comment. Posts remain visible after expiry but reject new interactions.

Phase	Requirement	Marks
A	Install and deploy in a virtualised environment; API reachable through VM IP; describe setup and folders	5
B	User management, JWT authentication and input validation	5
C	Complete Piazza REST API and data model	25
D	Demonstrate all 20 required test cases	15
E	Deploy with Docker in a Google Cloud VM or use DockerHub	10
F	Deploy to Kubernetes with a load balancer and five replicas	15
G	Technical report with design, endpoints, evidence and Harvard references	20
H	Readable, commented, well-structured code	5

Evidence rule: Screenshots must come from your own computer and Google Cloud account. Do not submit invented screenshots or claim that a deployment worked unless you personally ran it.

1.1 Functional checklist from the brief

- Only authenticated users can access Piazza resources.
- Users can register, log in and receive a JWT bearer token.
- Users can create posts with title, message, owner, timestamp, expiry and one or more allowed topics.
- Allowed topics are Politics, Health, Sport and Tech.
- Users can browse posts by topic.
- Users can like, dislike and comment on live posts.
- A post owner cannot like or dislike their own post.
- Expired posts remain visible but reject likes, dislikes and comments.
- Users can browse expired history by topic.
- Users can retrieve the live post with the greatest total likes plus dislikes.
- All 20 Olga/Nick/Mary/Nestor test cases are demonstrated.
- The application is containerised, deployed to a VM, then deployed to GKE with five replicas and a LoadBalancer Service.

2. Install the software on Windows

2.1 Install Visual Studio Code

1. Open a browser and search for the official Visual Studio Code download page.
2. Download the Windows installer.
3. Run the installer. Tick “Add to PATH” and “Add Open with Code action” if offered.
4. Open VS Code from the Start menu.

2.2 Install Node.js

Download the current Long-Term Support (LTS) installer from the official Node.js website. Accept the default options. Close and reopen VS Code afterwards.

```
node --version  
npm --version
```

Both commands must print version numbers. Node.js 18 or newer is required by the test runner; a current LTS release is preferred.

2.3 Install Git

Install Git for Windows using the default options, then verify:

```
git --version
```

2.4 Install Postman

Install the Postman desktop application. You will import the two JSON files found in the project postman folder.

2.5 Install Docker Desktop

5. Install Docker Desktop for Windows.
6. When asked, use the WSL 2 backend.
7. Restart Windows if prompted.
8. Open Docker Desktop and wait until it says the engine is running.

```
docker --version  
docker compose version
```

2.6 Install Google Cloud CLI

Install Google Cloud CLI for Windows from Google Cloud documentation. After installation, open a new PowerShell window and run:

```
gcloud --version
```

3. Open the supplied project and understand the folders

9. Download and extract Piazza_Beginner_Submission_Package.zip.
10. Open VS Code.
11. Click File > Open Folder.
12. Select the extracted Piazza_Beginner_Submission_Package folder.
13. Click Select Folder.
14. In VS Code, click Terminal > New Terminal.

```
npm install
```

The main structure is:

```
Piazza_Beginner_Submission_Package/
├─ app.js           Express application and routes
├─ server.js        Starts MongoDB and the HTTP server
├─ src/config        Database connection
├─ src/models        MongoDB User and Post schemas
├─ src/controllers   Application rules
├─ src/middleware    JWT, validation and errors
├─ src/routes        REST endpoint definitions
├─ tests            20-case automated runner
├─ postman           Importable Postman files
├─ kubernetes        Deployment, Service and Secret template
├─ Dockerfile        Instructions for building one image
├─ docker-compose.yml Local API + MongoDB containers
├─ docs              Report and this tutorial
└─ evidence          Your genuine screenshots
```

4. Create and connect MongoDB Atlas

MongoDB Atlas is the easiest database choice because the same database can be reached from your laptop, Docker container, Compute Engine VM and GKE cluster.

15. Open the MongoDB Atlas website and create an account.
16. Create a new project named Piazza Coursework.
17. Click Build a Database or Create a deployment.
18. Choose the free option where available and choose a nearby cloud region.
19. Create a database user. Use a unique username and a strong password. Save them privately.
20. Open Network Access or IP Access List.
21. For initial coursework testing, add 0.0.0.0/0 only if necessary for VM/GKE access. Atlas warns that this allows connections from anywhere, so the database username and password must remain strong. A safer alternative is to add only your current IP and later add cloud egress addresses.
22. Return to Database Deployments and click Connect.
23. Choose Drivers or Connect your application.
24. Select Node.js and copy the mongodb+srv connection string.
25. Replace <db_password> with your password and add /piazza before the question mark if a database name is not already present.

```
MONGODB_URI=mongodb+srv://YOUR_USER:YOUR_PASSWORD@YOUR_CLUSTER.mongodb.net/piazza?
retryWrites=true&w=majority
```

Password characters: If the password contains characters such as @, :, /, ? or #, URL-encode the password or create a simpler strong development password. Never publish the URI.

4.1 Create the private .env file

26. In VS Code Explorer, right-click .env.example.
27. Click Copy.
28. Right-click an empty area and click Paste.
29. Rename the copy to .env.
30. Open .env and enter your real values.

```
PORT=3000
MONGODB_URI=mongodb+srv://YOUR_USER:YOUR_PASSWORD@YOUR_CLUSTER.mongodb.net/piazza?
retryWrites=true&w=majority
JWT_SECRET=replace_with_a_long_random_private_string
JWT_EXPIRES_IN=2h
```

The .gitignore file excludes .env, so secrets should not be pushed to GitHub.

5. Build the code file by file

The project already contains these files. This section tells you exactly what belongs where and why. The full commented code remains in the downloadable project, so you can compare each file while typing or reviewing it.

app.js

Creates the Express application, security middleware, health endpoint and route groups.

```
// Load values from the private .env file before the application starts.
require('dotenv').config();

const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const morgan = require('morgan');

const authRoutes = require('./src/routes/authRoutes');
const postRoutes = require('./src/routes/postRoutes');
const { notFound, errorHandler } = require('./src/middleware/errorHandler');

const app = express();

// Add common security-related HTTP headers.
app.use(helmet());

// Allow tools such as Postman and a future front end to call the API.
app.use(cors());
```

```

// Read JSON request bodies and apply a sensible size limit.
app.use(express.json({ limit: '100kb' }));

// Print a short line in the terminal for each request during development.
app.use(morgan('dev'));

// A simple public endpoint used by Docker and Kubernetes health checks.
app.get('/health', (req, res) => {
  res.json({
    service: 'Piazza Cloud API',
    status: 'ok',
    time: new Date().toISOString()
  });
});

// Group related endpoints under clear REST-style paths.
app.use('/api/auth', authRoutes);
app.use('/api/posts', postRoutes);

// These must remain last so that valid routes run before error handling.
app.use(notFound);
app.use(errorHandler);

module.exports = app;

```

server.js

Checks configuration, connects to MongoDB and starts listening on port 3000.

```

// server.js is the entry point used by npm start and Docker.
require('dotenv').config();

const app = require('./app');
const connectDatabase = require('./src/config/database');

const port = Number(process.env.PORT || 3000);

// Stop immediately if the signing secret has not been configured.
if (!process.env.JWT_SECRET) {
  console.error('JWT_SECRET is missing. Copy .env.example to .env and edit it.');
```

```

  process.exit(1);
}

// Connect to MongoDB first. Only start accepting requests after it succeeds.
connectDatabase()
  .then(() => {
    app.listen(port, () => {
      console.log('Piazza API running at http://localhost:${port}');
```

```

    });
  })
  .catch((error) => {
    console.error('Application startup failed:', error.message);
    process.exit(1);
  });

```

src/config/database.js

Creates the Mongoose database connection.

```
const mongoose = require('mongoose');

// Open one reusable Mongoose connection for the whole application.
async function connectDatabase() {
  const uri = process.env.MONGODB_URI;

  if (!uri) {
    throw new Error('MONGODB_URI is not configured in the environment.');
```

```
  }
```

```
  await mongoose.connect(uri);
  console.log('MongoDB connected successfully: ${mongoose.connection.host}');
```

```
}
```

```
module.exports = connectDatabase;
```

src/models/User.js

Defines users and hashes passwords with bcrypt.

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

// This schema describes the documents stored in the users collection.
const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: true,
      trim: true,
      minlength: 2,
      maxlength: 50
    },
    email: {
      type: String,
      required: true,
      unique: true,
      lowercase: true,
      trim: true
    },
    // select:false stops the password hash appearing in normal database queries.
    passwordHash: {
      type: String,
      required: true,
      select: false
    }
  },
  { timestamps: true }
);
```

```
// Compare a login password with the stored bcrypt hash.
userSchema.methods.verifyPassword = function verifyPassword(password) {
  return bcrypt.compare(password, this.passwordHash);
};

// Keep password hashing inside the model so controllers stay easy to read.
userSchema.statics.createWithPassword = async function createWithPassword({
  name,
  email,
  password
}) {
  const passwordHash = await bcrypt.hash(password, 12);
  return this.create({ name, email, passwordHash });
};

module.exports = mongoose.model('User', userSchema);
```

src/models/Post.js

Defines posts, topics, expiry, likes, dislikes and comments.

```
const mongoose = require('mongoose');

// Likes and dislikes store who acted and when the action occurred.
const interactionSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true
    },
    userName: { type: String, required: true },
    createdAt: { type: Date, default: Date.now }
  },
  { _id: false }
);

// Comments have their own identifier and include the written text.
const commentSchema = new mongoose.Schema({
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  userName: { type: String, required: true },
  text: {
    type: String,
    required: true,
    trim: true,
    minlength: 1,
    maxlength: 500
  },
  createdAt: { type: Date, default: Date.now }
});
```



```

const postSchema = new mongoose.Schema(
  {
    title: {
      type: String,
      required: true,
      trim: true,
      minlength: 3,
      maxlength: 120
    },
    // The brief allows one post to belong to one or more of four topics.
    topics: [
      {
        type: String,
        enum: ['Politics', 'Health', 'Sport', 'Tech'],
        required: true
      }
    ],
    message: {
      type: String,
      required: true,
      trim: true,
      minlength: 3,
      maxlength: 2000
    },
    owner: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true
    },
    ownerName: { type: String, required: true },
    expiresAt: { type: Date, required: true },
    likes: { type: [interactionSchema], default: [] },
    dislikes: { type: [interactionSchema], default: [] },
    comments: { type: [commentSchema], default: [] }
  },
  {
    timestamps: true,
    toJSON: { virtuals: true },
    toObject: { virtuals: true }
  }
);

// Virtual values are calculated when data is returned; they are not duplicated in MongoDB.
postSchema.virtual('status').get(function status() {
  return new Date() >= this.expiresAt ? 'Expired' : 'Live';
});
postSchema.virtual('likeCount').get(function likeCount() {
  return this.likes.length;
});
postSchema.virtual('dislikeCount').get(function dislikeCount() {
  return this.dislikes.length;
});
postSchema.virtual('interestScore').get(function interestScore() {
  return this.likes.length + this.dislikes.length;
});

```

```
});  
  
module.exports = mongoose.model('Post', postSchema);
```

src/middleware/authenticate.js

Verifies bearer JWTs on every protected endpoint.

```
const jwt = require('jsonwebtoken');  
const User = require('../models/User');  
  
// Protect a route by checking the Authorization: Bearer <token> header.  
async function authenticate(req, res, next) {  
  try {  
    const header = req.headers.authorization || '';  
    const [scheme, token] = header.split(' ');  
  
    if (scheme !== 'Bearer' || !token) {  
      return res.status(401).json({ error: 'A Bearer token is required.' });  
    }  
  
    const payload = jwt.verify(token, process.env.JWT_SECRET);  
    const user = await User.findById(payload.sub);  
  
    if (!user) {  
      return res.status(401).json({ error: 'The token user no longer exists.' });  
    }  
  
    // Later controllers can now use req.user safely.  
    req.user = user;  
    next();  
  } catch (error) {  
    return res.status(401).json({ error: 'Invalid or expired token.' });  
  }  
}  
  
module.exports = authenticate;
```

src/middleware/validate.js

Returns clear input-validation errors.

```
const { validationResult } = require('express-validator');  
  
// Return all validation problems in one clear response.  
function validate(req, res, next) {  
  const result = validationResult(req);  
  
  if (!result.isEmpty()) {  
    return res.status(400).json({  
      error: 'Validation failed.',  
      details: result.array()  
    });  
  }  
}
```

```
    next();
  }

module.exports = validate;
```

src/middleware/errorHandler.js

Handles missing routes, duplicates and unexpected failures.

```
// Handle requests that do not match any endpoint.
function notFound(req, res) {
  res.status(404).json({
    error: `Route not found: ${req.method} ${req.originalUrl}`
  });
}

// Keep unexpected errors in one place instead of repeating try/catch responses.
function errorHandler(error, req, res, next) {
  console.error(error);

  // MongoDB uses error code 11000 for duplicate unique values.
  if (error.code === 11000) {
    return res.status(409).json({
      error: 'A record with that value already exists.'
    });
  }

  res.status(error.status || 500).json({
    error: error.message || 'Unexpected server error.'
  });
}

module.exports = { notFound, errorHandler };
```

src/controllers/authController.js

Implements registration, login and profile responses.

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

// Create a signed token that identifies one user for a limited time.
function issueToken(user) {
  return jwt.sign(
    { sub: user.id, name: user.name },
    process.env.JWT_SECRET,
    { expiresIn: process.env.JWT_EXPIRES_IN || '2h' }
  );
}

async function register(req, res, next) {
  try {
    const email = req.body.email.toLowerCase();
```

```

const existingUser = await User.findOne({ email });

if (existingUser) {
  return res.status(409).json({ error: 'Email is already registered.' });
}

const user = await User.createWithPassword({ ...req.body, email });

res.status(201).json({
  user: { id: user.id, name: user.name, email: user.email },
  token: issueToken(user)
});
} catch (error) {
  next(error);
}
}

async function login(req, res, next) {
  try {
    const email = req.body.email.toLowerCase();
    const user = await User.findOne({ email }).select('+passwordHash');

    if (!user || !(await user.verifyPassword(req.body.password))) {
      return res.status(401).json({ error: 'Incorrect email or password.' });
    }

    res.json({
      user: { id: user.id, name: user.name, email: user.email },
      token: issueToken(user)
    });
  } catch (error) {
    next(error);
  }
}

function profile(req, res) {
  res.json({
    user: {
      id: req.user.id,
      name: req.user.name,
      email: req.user.email
    }
  });
}

module.exports = { register, login, profile };

```

src/controllers/postController.js

Implements post creation, browsing, expiry, reactions and comments.

Open this file in the included project. It is longer, so read the helper functions first, then each controller method. Comments at the top explain the overall flow.

src/routes/authRoutes.js

Maps authentication URLs to controllers and validation rules.

```
const router = require('express').Router();
const { body } = require('express-validator');

const { register, login, profile } = require('../controllers/authController');
const authenticate = require('../middleware/authenticate');
const validate = require('../middleware/validate');

// Public registration route with server-side validation.
router.post(
  '/register',
  [
    body('name').trim().isLength({ min: 2, max: 50 }),
    body('email').isEmail().normalizeEmail(),
    body('password').isLength({ min: 8, max: 72 })
  ],
  validate,
  register
);

// Public login route. A successful response contains a JWT.
router.post(
  '/login',
  [body('email').isEmail().normalizeEmail(), body('password').notEmpty()],
  validate,
  login
);

// This route demonstrates that the JWT middleware protects resources.
router.get('/profile', authenticate, profile);

module.exports = router;
```

src/routes/postRoutes.js

Maps all protected Piazza URLs to controllers.

```
const router = require('express').Router();
const { body, param } = require('express-validator');

const authenticate = require('../middleware/authenticate');
const validate = require('../middleware/validate');
const postController = require('../controllers/postController');

const allowedTopics = ['Politics', 'Health', 'Sport', 'Tech'];

// Every endpoint below this line requires a valid JWT.
router.use(authenticate);

router.post(
  '/',
  [
```

```

    body('title').trim().isLength({ min: 3, max: 120 }),
    body('message').trim().isLength({ min: 3, max: 2000 }),
    body('topics').isArray({ min: 1 }),
    body('topics.*').isIn(allowedTopics),
    body('expiresAt').isISO8601()
  ],
  validate,
  postController.createPost
);

router.get('/', postController.listPosts);

// Put specific topic routes before /:postId so Express matches them correctly.
router.get(
  '/topic/:topic/expired',
  [param('topic').isIn(allowedTopics)],
  validate,
  postController.expiredByTopic
);
router.get(
  '/topic/:topic/most-active',
  [param('topic').isIn(allowedTopics)],
  validate,
  postController.mostActive
);
router.get(
  '/topic/:topic',
  [param('topic').isIn(allowedTopics)],
  validate,
  postController.byTopic
);

router.get('/:postId', postController.getPost);
router.post('/:postId/like', postController.like);
router.post('/:postId/dislike', postController.dislike);
router.post(
  '/:postId/comments',
  [body('text').trim().isLength({ min: 1, max: 500 })],
  validate,
  postController.comment
);

module.exports = router;

```

5.1 package.json

```

{
  "name": "piazza-cloud-api",
  "version": "1.0.0",
  "description": "Original RESTful Piazza SaaS coursework project",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js",
    "test": "jest --runInBand",
  }
}

```

```
"test:coursework": "node tests/courseworkTestRunner.js"
},
"engines": {"node": ">=18"},
"dependencies": {
  "bcryptjs": "^2.4.3",
  "cors": "^2.8.5",
  "dotenv": "^16.4.5",
  "express": "^4.19.2",
  "express-validator": "^7.2.0",
  "helmet": "^7.1.0",
  "jsonwebtoken": "^9.0.2",
  "mongoose": "^8.5.3",
  "morgan": "^1.10.0"
},
"devDependencies": {
  "jest": "^29.7.0",
  "nodemon": "^3.1.4",
  "supertest": "^7.0.0"
}
}
```

Run npm install whenever package.json dependencies change. Use npm run dev during development and npm start for normal execution.

6. Run the API locally

31. Open the project in VS Code.
32. Click Terminal > New Terminal.
33. Run npm install once.
34. Confirm .env contains the Atlas URI and JWT secret.
35. Run npm run dev.

```
npm install
npm run dev
```

Expected terminal messages:

```
MongoDB connected successfully: ...
Piazza API running at http://localhost:3000
```

36. Open a browser.
37. Visit <http://localhost:3000/health>.
38. Confirm the response contains status “ok”.
39. Capture a screenshot showing the browser response and the terminal. Save it in evidence/local/.

7. Test manually with Postman

7.1 Import the supplied collection

40. Open Postman.
41. Click Import.
42. Choose Files.
43. Select postman/Piazza_API.postman_collection.json and postman/Piazza.postman_environment.json.
44. Click Import.
45. Use the environment selector in the top-right and choose Piazza Local.

7.2 Register a user

Create or open a POST request to:

```
POST http://localhost:3000/api/auth/register
```

Choose Body > raw > JSON and enter:

```
{
  "name": "Olga",
  "email": "olga@example.com",
  "password": "Coursework123!"
}
```

Click Send. Expect HTTP 201 and a token.

7.3 Login and store the token

```
POST http://localhost:3000/api/auth/login
```

```
{
  "email": "olga@example.com",
  "password": "Coursework123!"
}
```

The supplied collection stores the returned token automatically. For a manual request, open Authorization, choose Bearer Token and paste the token.

7.4 Create a post

```
POST http://localhost:3000/api/posts
```

```
{
  "title": "Cloud services for students",
  "topics": ["Tech"],
  "message": "This is my original Piazza post about cloud services.",
  "expiresAt": "2030-12-31T23:59:59.000Z"
}
```

Use a future ISO date. Expect HTTP 201.

7.5 Endpoint table

Method	Endpoint	Expected purpose
POST	/api/auth/register	Create a user
POST	/api/auth/login	Receive JWT
GET	/api/auth/profile	Prove token works
POST	/api/posts	Create post
GET	/api/posts	Browse all
GET	/api/posts/:postId	Browse one
GET	/api/posts/topic/:topic	Browse topic
POST	/api/posts/:postId/like	Like live post
POST	/api/posts/:postId/dislike	Dislike live post
POST	/api/posts/:postId/comments	Add comment
GET	/api/posts/topic/:topic/expired	Expired history
GET	/api/posts/topic/:topic/most-active	Highest likes + dislikes

8. Demonstrate every one of the 20 required test cases

Keep the API running in the first terminal. Open Terminal > New Terminal to create a second terminal, then run:

```
npm run test:coursework
```

The runner creates unique users on every execution and performs all required actions. A successful finish is:

```
Result: 20 passed, 0 failed.
```

Test	What the automated runner proves
TC1	Register Olga, Nick, Mary and Nestor.
TC2	Log in all four users and receive tokens.
TC3	Olga calls a protected endpoint without a token and receives 401.
TC4	Olga creates a Tech post with expiry.
TC5	Nick creates a Tech post.
TC6	Mary creates a Tech post.
TC7	Nick and Olga browse three untouched Tech posts.
TC8	Nick and Olga like Mary's post.
TC9	Nestor likes Nick's post and dislikes Mary's post.
TC10	Nick confirms Mary has two likes/one dislike and Nick has one like.
TC11	Mary is prevented from liking her own post.
TC12	Nick and Olga add two comments each in round-robin order.
TC13	Nick sees the comments and interaction totals.
TC14	Nestor creates a short-lived Health post.
TC15	Mary browses Health and sees Nestor's post.
TC16	Mary comments before expiry.

TC17	Mary's dislike after expiry fails.
TC18	Nestor sees the expired Health post with one comment.
TC19	Nick sees an empty expired Sports result.
TC20	Nestor receives Mary's post as the most-active Tech post.

Evidence: Capture the terminal from the first PASS line through the final result. For a readable report, use two or three screenshots rather than shrinking one very tall image.

9. Put the project in your own GitHub repository

46. Sign in to GitHub.
47. Click the + button > New repository.
48. Name it piazza-cloud-api-yourstudentid.
49. Keep it private or public according to your submission rules.
50. Do not initialise it with another README if the local project already contains one.
51. Click Create repository.

In the VS Code terminal run the commands GitHub shows, or use:

```
git init
git add .
git commit -m "Create original Piazza cloud API"
git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/YOUR_REPOSITORY.git
git push -u origin main
```

Before pushing, run git status and confirm .env and kubernetes/secret.yaml are not listed.

```
git status
```

10. Build and run Docker containers locally

10.1 Understand the Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
ENV NODE_ENV=production
EXPOSE 3000
USER node
CMD ["node", "server.js"]
```

The Dockerfile builds one production API image. It copies package files first for better build caching, installs production dependencies, copies source code, exposes port 3000 and starts server.js as the non-root node user.

10.2 Use Docker Compose for API plus local MongoDB

```
services:
  mongo:
    image: mongo:7
    restart: unless-stopped
    volumes:
      - piazza_mongo_data:/data/db
  api:
    build: .
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      PORT: 3000
      MONGODB_URI: mongodb://mongo:27017/piazza
      JWT_SECRET: local_docker_secret_change_before_deployment
      JWT_EXPIRES_IN: 2h
    depends_on:
      - mongo
    volumes:
      piazza_mongo_data:
```

52. Open Docker Desktop and wait for it to run.
53. In VS Code terminal, stop npm with Ctrl+C if necessary.
54. Run `docker compose up --build`.
55. Wait until both MongoDB and the API report that they are ready.
56. Open `http://localhost:3000/health`.
57. Run `npm run test:coursework` in another terminal.

```
docker compose up --build
```

Useful checks:

```
docker compose ps
docker images
docker volume ls
```

Stop without deleting data:

```
docker compose down
```

Stop and delete the local MongoDB volume only when you intentionally want a clean database:

```
docker compose down -v
```

11. Publish the Docker image

Option A: Docker Hub

58. Create a Docker Hub account and repository named piazza-cloud-api.
59. In PowerShell run docker login.
60. Build the image using your real Docker Hub username.
61. Push the image.

```
docker login
docker build -t YOUR_DOCKERHUB_USERNAME/piazza-cloud-api:1.0 .
docker push YOUR_DOCKERHUB_USERNAME/piazza-cloud-api:1.0
```

Option B: Google Artifact Registry

Artifact Registry is Google Cloud's native container registry. It avoids relying on Docker Hub and integrates directly with GKE. Follow the official Artifact Registry setup for your chosen region, then tag and push the image to the generated registry path.

12. Deploy the Docker container to a Google Cloud VM (Phases A and E)

12.1 Create the Google Cloud project

62. Open Google Cloud Console.
63. Use the project selector at the top and click New Project.
64. Name it Piazza Coursework and create it.
65. Attach billing if required.
66. Open APIs & Services > Library.
67. Enable Compute Engine API.

12.2 Create the VM

68. Open Navigation menu > Compute Engine > VM instances.
69. Click Create instance.
70. Name it piazza-vm.
71. Choose a nearby region and zone.
72. Choose a small Linux machine type suitable for coursework.
73. Use an Ubuntu LTS boot disk.
74. Tick Allow HTTP traffic.
75. Click Create.

12.3 Connect and install Docker

On the VM instances page, click SSH beside piazza-vm. A browser terminal opens. Install Docker by following Docker's official Ubuntu instructions. The exact repository commands may change, so use the current official page rather than copying an old tutorial blindly.

After installation, verify:

```
docker --version
```

12.4 Run your published image

Use Atlas for the database. Create an env file on the VM without committing it:

```
cat > .env <<'EOF'
PORT=3000
MONGODB_URI=YOUR_REAL_ATLAS_URI
JWT_SECRET=YOUR_REAL_LONG_SECRET
JWT_EXPIRES_IN=2h
EOF
```

Pull and run the container:

```
docker pull YOUR_DOCKERHUB_USERNAME/piazza-cloud-api:1.0
docker run -d \
  --name piazza-api \
  --restart unless-stopped \
  --env-file .env \
  -p 3000:3000 \
  YOUR_DOCKERHUB_USERNAME/piazza-cloud-api:1.0
```

```
docker ps
docker logs piazza-api
```

12.5 Create a firewall rule for port 3000

76. In Google Cloud Console open VPC network > Firewall.
77. Click Create firewall rule.
78. Name it allow-piazza-3000.
79. Direction: Ingress; Action: Allow.
80. Targets: all instances or use a network tag such as piazza-api.
81. Source IPv4 ranges: use your own public IP for safer testing, or 0.0.0.0/0 only when public coursework access is required.
82. Protocols and ports: Specified protocols and ports > TCP > 3000.
83. Create the rule.

Open the VM's External IP followed by :3000/health:

```
http://VM_EXTERNAL_IP:3000/health
```

Phase A/E evidence: Capture the VM list with external IP, SSH terminal showing docker ps, docker logs showing MongoDB connection, browser/Postman calling the external IP, and the GitHub repository page. Do not paste long Linux command listings into sections where the brief says not to; present a concise deployment explanation and selected commands only where Phase E explicitly asks for them.

13. Deploy five replicas to Google Kubernetes Engine (Phase F)

13.1 Enable services and create the cluster

84. In Google Cloud Console enable Kubernetes Engine API.
85. Open Kubernetes Engine > Clusters.
86. Click Create.
87. Choose Autopilot for a simpler managed cluster or Standard if your tutorial requires node-level choices.
88. Name the cluster piazza-cluster.
89. Choose a nearby region.
90. Create the cluster and wait until its status is ready.

Open Cloud Shell using the terminal icon at the top-right of Google Cloud Console. Select the correct project:

```
gcloud config set project YOUR_PROJECT_ID
```

On the cluster page click Connect and copy the displayed gcloud get-credentials command. Run it in Cloud Shell. Then verify:

```
kubectl get nodes
```

13.2 Edit the Kubernetes deployment

Open kubernetes/deployment.yaml locally and replace YOUR_DOCKERHUB_USERNAME. The required replica count is already five:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: piazza-api
spec:
  replicas: 5
  selector:
    matchLabels:
      app: piazza-api
  template:
    metadata:
      labels:
        app: piazza-api
    spec:
      containers:
        - name: piazza-api
          image: YOUR_DOCKERHUB_USERNAME/piazza-cloud-api:1.0
          ports:
            - containerPort: 3000
          env:
            - name: PORT
              value: "3000"
            - name: MONGODB_URI
```

```

    valueFrom:
      secretKeyRef:
        name: piazza-secrets
        key: mongodb-uri
  - name: JWT_SECRET
    valueFrom:
      secretKeyRef:
        name: piazza-secrets
        key: jwt-secret
  readinessProbe:
    httpGet:
      path: /health
      port: 3000
    initialDelaySeconds: 5
    periodSeconds: 10

```

13.3 Create the Kubernetes Secret safely

Do not commit a real secret YAML. The safest beginner command creates the Secret directly from literal values:

```

kubectl create secret generic piazza-secrets \
--from-literal=mongodb-uri='YOUR_REAL_ATLAS_URI' \
--from-literal=jwt-secret='YOUR_REAL_LONG_SECRET'

```

Use single quotes around the values in Cloud Shell. Confirm only the Secret name, not its decoded contents:

```

kubectl get secret piazza-secrets

```

13.4 Upload and apply YAML

Either clone your GitHub repository in Cloud Shell or use the Cloud Shell file upload button. From the project folder run:

```

kubectl apply -f kubernetes/deployment.yaml
kubectl apply -f kubernetes/service.yaml
kubectl get deployments
kubectl get pods
kubectl get services

```

The Service is type LoadBalancer:

```

apiVersion: v1
kind: Service
metadata:
  name: piazza-load-balancer
spec:
  type: LoadBalancer
  selector:
    app: piazza-api
  ports:
    - protocol: TCP
      port: 80

```

```
targetPort: 3000
```

Wait until EXTERNAL-IP changes from pending to an address. Then test:

```
http://LOAD_BALANCER_EXTERNAL_IP/health
```

13.5 Prove there are five replicas

```
kubectl get deployment piazza-api
kubectl get pods -l app=piazza-api -o wide
```

The deployment should show 5 desired and 5 available replicas. Capture the output and the GKE Workloads page.

13.6 Troubleshoot and clean up

```
kubectl describe pod POD_NAME
kubectl logs POD_NAME
kubectl describe service piazza-load-balancer
```

To avoid ongoing charges after collecting evidence:

```
gcloud container clusters delete piazza-cluster --region YOUR_REGION
```

Cost warning: GKE clusters, load balancers, VM instances and external IPs can incur charges. Delete resources after your evidence is complete unless your lecturer requires them to remain online until marking.

14. Technical report: exact recommended structure

Report section	What to include
Title page	Module, coursework title, your name, student ID, date and GitHub link.
1. Introduction	Purpose of Piazza and summary of the chosen technology.
2. Requirements and design	Map the six Piazza actions to endpoints; include an architecture diagram.
3. Phase A setup	VM, installed software and folder structure. Follow the brief's instruction about command listings.
4. Phase B security	Registration, bcrypt password hashing, JWT middleware, validation and HTTP errors.
5. Phase C API	User/Post schemas, expiry rule, endpoint table and example requests/responses.
6. Phase D testing	Explain all 20 test cases and include readable evidence.
7. Phase E Docker/VM	Dockerfile, image, container, external VM endpoint and selected deployment evidence.
8. Phase F Kubernetes	Deployment with replicas: 5, Secret, readiness

	probe and LoadBalancer Service.
9. Complexity and quality	Discuss operations such as database queries and most-active sorting; explain modular design and comments.
10. Limitations and future work	Role-based moderation, refresh tokens, pagination, indexes, CI/CD, Terraform and front end.
11. Conclusion	Summarise what was achieved.
References	Harvard references for tutorial repository, Node.js, Express, MongoDB, Docker and Google Cloud documentation.
Appendix	Selected code, full endpoint list, test output and additional screenshots.

14.1 Architecture diagram to draw

```

Postman / automated test runner
|
v
Google Cloud Load Balancer or VM external IP
|
v
Five Piazza API containers/pods (Node.js + Express)
|
v
MongoDB Atlas database

GitHub -> Docker image registry -> VM/GKE deployment

```

15. Complexity discussion for Phase H

- User lookup by unique email is normally efficient when MongoDB maintains a unique index: approximately $O(\log n)$ index lookup in simplified terms.
- Browsing posts by topic depends on the number of matching posts returned; response construction is $O(k)$ for k returned posts.
- Checking whether one user already reacted currently scans the reaction array, $O(r)$, where r is the number of reactions on that post.
- The current most-active endpoint loads live posts for a topic and sorts them, $O(p \log p)$ for p live posts. A MongoDB aggregation pipeline or stored interaction score could improve large-scale performance.
- Adding a comment is effectively $O(1)$ application work plus database document update, although a very large embedded comment array would eventually require redesign or pagination.

16. Final submission checklist

- ☐ Application runs locally and `/health` returns ok.
- ☐ MongoDB Atlas contains users and posts created by your own test run.

- ☐ Unauthorised requests receive 401.
- ☐ All post routes require JWT.
- ☐ All four allowed topics work and invalid topics are rejected.
- ☐ Post owner cannot like/dislike their own post.
- ☐ Expired posts reject interactions and remain visible.
- ☐ Expired history and most-active endpoints work.
- ☐ Automated runner reports 20 passed, 0 failed.
- ☐ Postman collection is included.
- ☐ .env and real Kubernetes secrets are not in GitHub.
- ☐ Docker image builds and container works.
- ☐ VM endpoint works through external IP.
- ☐ GKE deployment displays five available replicas.
- ☐ LoadBalancer has an external IP and /health works.
- ☐ Report contains your own screenshots, architecture diagram, endpoint table, testing discussion and Harvard references.
- ☐ GitHub commit history shows gradual work rather than one unexplained upload.
- ☐ README has exact run instructions.
- ☐ Cloud resources are deleted after evidence is collected if they are no longer needed.

17. Common problems and fixes

Problem	Fix
node is not recognised	Restart VS Code after Node installation; verify PATH; reinstall the LTS installer.
MongoServerError	Check Atlas username/password, connection string and IP Access List.
JWT_SECRET is missing	Create .env in the project root and restart npm.
401 in Postman	Use Authorization > Bearer Token and the newest login token.
expiresAt must be in the future	Use a future ISO 8601 date.
Docker daemon is not running	Open Docker Desktop and wait for the engine.
Port 3000 already in use	Stop the other process/container or change the host port.
GKE ImagePullBackOff	Check image name, tag and registry visibility/permissions.

GKE pods CrashLoopBackOff	Run kubectl logs; verify Secret names and MongoDB network access.
LoadBalancer external IP pending	Wait several minutes; inspect kubectl describe service; confirm billing/API availability.
Git push rejected	Run git pull --rebase origin main, resolve conflicts, then push; do not force-push unless you understand the consequence.

18. References and source acknowledgement

Use Harvard style in the report. Suggested source categories:

- Coursework brief: Cloud Computing Coursework - Development of a Cloud Software as a Service (Piazza).
- warestack/cc GitHub tutorial repository, citing the specific class folders used.
- Node.js official documentation.
- Express official documentation.
- MongoDB Atlas and Mongoose documentation.
- Docker official Node.js and Dockerfile guides.
- Google Cloud Compute Engine documentation.
- Google Kubernetes Engine quickstart and service exposure documentation.
- JWT and bcrypt package documentation where implementation details are discussed.

In source files, short comments may acknowledge an adapted pattern, for example: “Routing pattern informed by the module Express tutorial; endpoint design and Piazza business rules are original.” Do not copy tutorial code without attribution.

19. What you must personalise before submission

- Change sample post titles/messages so they reflect your own project demonstration.
- Add your student name and ID to the report only, not into secret values.
- Use your own repository URL, Docker image name, Google Cloud project ID, region and external IPs.
- Insert your own screenshots and captions.
- Rewrite report explanations in words you genuinely understand and can defend in a viva.
- Run every command and verify every claim yourself.

Final reminder: This package gives you a working starting point and a complete route through the marking criteria. Submission readiness still depends on your own successful runs, screenshots, cloud identifiers, report personalisation and institutional academic-integrity rules.